

Ghid CompletableFuture (Java 8+) - Cheat Sheet

CompletableFuture<T> este un container pentru un rezultat care nu este încă disponibil. Acesta permite înlănțuirea operațiilor asincrone fără a bloca thread-ul principal (Main Thread).

1. Creare (Pornire Task Asincron)

Cel mai des vei folosi `supplyAsync` pentru operații care returnează date (ex: interogări în baza de date).

```
// 1. Fără return (doar execută ceva în background)
CompletableFuture<Void> voidFuture = CompletableFuture.runAsync(() -> {
    System.out.println("Rulează pe un thread worker: " + Thread.currentThread().getName());
});
```

```
// 2. Cu return (Cel mai util pentru Repository)
CompletableFuture<List<Student>> future = CompletableFuture.supplyAsync(() -> {
    // Aici apelezi Repository-ul (operație blocantă)
    return repo.findAll();
});
```

```
// 3. Cu Executor Custom (Recomandat pentru controlul thread-urilor)
ExecutorService executor = Executors.newFixedThreadPool(2);
CompletableFuture.supplyAsync(() -> repo.findAll(), executor);
```

2. Prelucrare Rezultat (Chaining)

Aceste metode se execută automat după ce task-ul anterior s-a finalizat cu succes.

| **Metodă** | **Semnătură Funcțională** | **Descriere** | **Utilizare Tipică** |

| **thenApply** | T -> U (Function) | Transformă rezultatul. Returnează un nou Future. | Mapare date (ex: Entity -> DTO). |

| **thenAccept** | T -> void (Consumer) | Consumă rezultatul. Nu returnează nimic. | Actualizare UI, afișare consolă. |

| **thenRun** | () -> void (Runnable) | Rulează cod după finalizare. Ignoră rezultatul. | Logare

"Gata", curățare resurse. |

Exemplu:

```
CompletableFuture.supplyAsync(() -> repo.findById(1)) // Returnează Student
    .thenApply(student -> student.getNum().toUpperCase()) // Transformă în String
    .thenAccept(ume -> System.out.println("Nume procesat: " + ume)); // Printează
```

3. Integrare cu JavaFX (CRITIC)

Deoarece `supplyAsync` rulează pe un thread de tip *worker*, **NU** poți modifica componentele UI (`TableView`, `Label`, `Button`) direct din `thenAccept`. Trebuie să revii pe thread-ul aplicației.

```
service.findAllAsync() // Returnează CompletableFuture<List<Student>>
    .thenAccept(studentList -> {
        // AICI: Suntem încă pe thread-ul de background!

        Platform.runLater(() -> {
            // AICI: Suntem pe JavaFX Application Thread
            model.setAll(studentList);
            statusLabel.setText("Date încărcate cu succes!");
        });
    });
```

4. Combinarea a două Future-uri

Util când ai nevoie de date din surse diferite și vrei să aștepti rezultatele.

A. `thenCompose` (`FlatMap`)

Folosit când rezultatul primului Future este necesar pentru a porni al doilea (Dependență: **A** -> apoi **B**).

```
getUserById(1) // returnează Future<User>
    .thenCompose(user -> getGradesByUser(user)) // returnează Future<List<Grade>>
    .thenAccept(grades -> System.out.println(grades));
```

B. thenCombine (AND)

Folosit când cele două Future-uri pot rula în paralel, dar ai nevoie de ambele la final (Independent: **A și B simultan**).

```
CompletableFuture<User> userFut = CompletableFuture.supplyAsync(() ->
repoUser.findById(1));
CompletableFuture<List<Nota>> noteFut = CompletableFuture.supplyAsync(() ->
repoNote.findAll());

userFut.thenCombine(noteFut, (user, note) -> {
    return "Userul " + user.getNume() + " are " + note.size() + " note.";
}).thenAccept(rezultat -> System.out.println(rezultat));
```

5. Gestionarea Erorilor (Exception Handling)

Dacă o etapă aruncă o excepție, tot lanțul se oprește. Folosește exceptionally pentru fallback.

```
CompletableFuture.supplyAsync(() -> {
    if (true) throw new RuntimeException("Eroare DB!");
    return "Succes";
})
.exceptionally(ex -> {
    // Se execută DOAR dacă a apărut o eroare
    System.err.println("A crăpat: " + ex.getMessage());
    return "Valoare Default"; // Returnezi o valoare de fallback
})
.thenAccept(val -> System.out.println(val)); // Va printa "Valoare Default"
```

6. Async vs Sync Methods

- **thenApply**: Se execută pe același thread care a finalizat task-ul anterior. Este mai rapid (fără context switch).
- **thenApplyAsync**: Trimite execuția înapoi în Executor (alt thread din pool). Util pentru procesări foarte grele.

La examen, varianta simplă (fără suffix-ul Async pe metodele de înlănțuire) este de obicei suficientă.

7. Obținerea Valorii (Blocant)

Folosit în main sau în teste pentru a aștepta rezultatul:

- **join()**: Returnează rezultatul sau aruncă `UncheckedException`. (Preferat).
- **get()**: Returnează rezultatul sau aruncă `CheckedException` (necesită try-catch).

```
String result = future.join(); // Blochează execuția până e gata
```

8. Exemplu Complet (Pattern de Examen)

Șablon recomandat pentru metodele din Controller:

```
public void handleLoadData() {  
    // 1. Feedback Vizual  
    loadingSpinner.setVisible(true);  
  
    // 2. Start Async  
    service.getAllEntityAsync()  
        .thenApply(list -> {  
            // (Optional) Filtrare/Sortare grea pe background thread  
            return list.stream()  
                .filter(e -> e.isActive())  
                .collect(Collectors.toList());  
        })  
        .exceptionally(ex -> {  
            // Tratare erori pe UI  
            Platform.runLater(() -> showErrorAlert("Eroare la citire: " + ex.getMessage()));  
            return new ArrayList<>(); // Returnăm listă goală pentru a continua sigur  
        })  
        .thenAccept(list -> {  
            // 3. Update UI  
            Platform.runLater(() -> {  
                model.setAll(list);  
                loadingSpinner.setVisible(false);  
            });  
        });  
}
```